(if they are modified)	chemical in the balanced equation. molar_mass – Float: Stores the molar mass of the chemical in grams per mole. mass – Float: Stores the mass of the chemical entered by the user, measured in grams. is_reactant – Boolean: Identifies whether the chemical is a reactant or a product.
Routes (minimum 7) The names of your route functions	home – The homepage will welcome users and explain the purpose of the website. enter_equation – Allows the user to enter a balanced chemical equation. enter_reactants – Allows the user to enter the masses of each reactant. enter_products – Allows the user to enter the actual mass of the product produced. results – Displays the limiting reactant, theoretical yield, and percent yield. explanation – Explains how the program calculated its answers, including the steps involved. help – Provides instructions and examples to help users understand how to use the website.
Pages (minimum 5) A brief description of the pages in your site	Home Page: Welcomes the user and explains the purpose of the website. It will include a button to begin a new chemical reaction calculation. Chemical Equation Page: Allows the user to enter an already balanced chemical equation. Reactant Input Page: Allows the user to enter the mass of each reactant in grams. Product Input Page: Allows the user to enter the actual yield of the product in grams. Results Page: Displays the limiting reactant, theoretical yield, and percent yield.

Site Functionality Requirements

Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

- At least 7 routes.
- At least 5 pages.
- The implementation of the *State* dataclass must have at least 4 fields.
  - At least one field must be a list of class instances or a dictionary of class instances.
    - This means that there must be a second dataclass.
  - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (global constants are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above requirements. Further, **if you miss more than 2 requirements, you will get no credit for that submission**. For example, missing a second dataclass means you would be missing not only that requirement, but also the list or dictionary of such dataclass instances, and the loop iterating over that list. This would mean that the submission would get a zero.

Code Requirements

Make sure your program and website runs; **broken programs do not earn points**. This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any other penalties listed in the syllabus.

Once you are finished with your project code, check over the following:

- Chosen good variable names (check last two sections of Chapter 1B1)
- Unit tested your functions where possible, being valid and thorough: each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
  - If a function can't be unit tested, you must note as much in a docstring or comment.
- Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
- Included types in your function headers
- Included good docstrings as the first line of your function's body
- Decomposed complicated functions with helper functions

textbook?page=bakery_functions...

I am making a coding website project for my computer science class. we are allowed to have AI make the code for us. we are using thoney and having the website deployed using drafter. Using these requirments and plans make the code for me

Worked for 1m 22s >

Based on your project plan and the assignment requirements, I would build this as a **Stoichiometry Calculator Website** using Python, Thonny, and the Drafter library.

The website will let students:

1. Enter a balanced chemical equation.
2. Enter the starting mass of each reactant.
3. Enter the actual amount of product collected.
4. Calculate the limiting reactant.
5. Calculate theoretical yield and percent yield.
6. View an explanation of how each answer was calculated.

I'll organize the code around your two dataclasses (`Reaction` and `Chemical`) and your seven planned routes. I'll also make sure the structure addresses the assignment's requirements for state changes, user inputs, loops, and conditional statements.

1. Complete Python code

This version uses:

- `Reaction`? Your planned main dataclass will be called `State`.
- `Chemical` as your second dataclass.
- A list of `Chemical` objects stored in `State`.
- Loops to calculate the limiting reactant.
- Conditional statements to validate information and calculate results.
- Drafter's `@route`, `Page`, `TextBox`, `Button`, and `Link` components.

One design decision: Students will manually enter each chemical's formula, coefficient, molar mass, and mass. This avoids needing a complicated chemical equation parser or a large database of chemical information. The entered chemicals must match the balanced equation.

The first version calculates the yield of one selected product at a time. Students can enter multiple reactants.

Copy the following into a new Python file in Thonny.

```
from drafter import *
from dataclasses import dataclass


# ---------------------------------------
# DATACLASSES
# ---------------------------------------

@dataclass
class Chemical:
    name: str
    coefficient: int
    molar_mass: float
    mass: float
    is_reactant: bool


@dataclass
class State:
    equation: str
    chemicals: list[Chemical]
    limiting_reactant: str
    theoretical_yield: float
    actual_yield: float
    percent_yield: float
    error_message: str
```

```
# ----------------------------------------
# HELPER FUNCTIONS
# ----------------------------------------

def chemical_list(state: State, reactants: bool) -> list:
    """Creates a list of chemicals matching the selected type."""
    result = []

    for chemical in state.chemicals:
        if chemical.is_reactant == reactants:
            result.append(
                chemical.name + " - " +
                str(chemical.mass) + " grams"
            )

    return result


def calculate_values(state: State) -> bool:
    """Calculates the limiting reactant and product yields."""

    reactants = []
    products = []

    state.error_message = ""

    # Separate reactants and products.
    for chemical in state.chemicals:
        if chemical.is_reactant:
            reactants.append(chemical)
        else:
            products.append(chemical)

    # Check that the required chemicals exist.
    if len(reactants) == 0:
        state.error_message = "Please enter at least one reactant."
        return False

    if len(products) != 1:
        state.error_message = "Please enter exactly one product."
        return False
```

```
# Check reactant information.
for chemical in reactants:
    if (chemical.molar_mass <= 0 or
            chemical.coefficient <= 0 or
            chemical.mass < 0):
        state.error_message = "Check your reactant information."
        return False

product = products[0]

if (product.molar_mass <= 0 or
        product.coefficient <= 0 or
        product.mass < 0):
    state.error_message = "Check your product information."
    return False

# Find the limiting reactant.
smallest_ratio = None
limiting_name = ""

for chemical in reactants:
    moles = chemical.mass / chemical.molar_mass

    ratio = moles / chemical.coefficient

    if smallest_ratio is None or ratio < smallest_ratio:
        smallest_ratio = ratio
        limiting_name = chemical.name

state.limiting_reactant = limiting_name

# Calculate theoretical yield.
product_moles = smallest_ratio * product.coefficient

state.theoretical_yield = (
    product_moles * product.molar_mass
)

# Store actual yield.
state.actual_yield = product.mass

# Calculate percent yield.
if state.theoretical_yield > 0:
    state.percent_yield = (
```

```
            state.actual_yield / state.theoretical_yield
        ) * 100
    else:
        state.error_message = (
            "The theoretical yield is zero. "
            "Check your reactant masses."
        )
        return False

    return True


# ----------------------------------------
# HOME PAGE
# ----------------------------------------

@route
def index(state: State) -> Page:
    return Page(state, [
        Header("Stoichiometry Calculator"),

        "Welcome to the Stoichiometry Calculator!",

        "This website helps students solve stoichiometry "
        "problems by calculating the limiting reactant, "
        "theoretical yield, and percent yield.",

        Header("Get Started", 2),

        Button("Start a New Calculation", "enter_equation"),

        Link("Help and Instructions", "help"),
        "\n",
        Link("Learn How Calculations Work", "explanation")
    ])


# Home route required by project plan.
@route
def home(state: State) -> Page:
    return index(state)


# ----------------------------------------
```

```
# EQUATION INPUT PAGE
# ----------------------------------------

@route
def enter_equation(state: State) -> Page:
    return Page(state, [
        Header("Enter Chemical Equation"),

        "Enter an already balanced chemical equation.",

        "Example: 2 H2 + O2 -> 2 H2O",

        "Balanced Chemical Equation:",

        TextBox("equation", state.equation),

        Button("Save Equation", "save_equation")
    ])


@route
def save_equation(state: State, equation: str) -> Page:

    if equation.strip() == "":
        state.error_message = "Please enter an equation."
        return Page(state, [
            state.error_message,
            Link("Return to equation input", "enter_equation")
        ])

    state.equation = equation
    state.chemicals = []
    state.limiting_reactant = ""
    state.theoretical_yield = 0.0
    state.actual_yield = 0.0
    state.percent_yield = 0.0
    state.error_message = ""

    return Page(state, [
        Header("Equation Saved!"),

        "Your balanced equation:",

        state.equation,
```

```
        Button("Enter Reactants", "enter_reactants"),

        Link("Home", "home")
    ])


# ----------------------------------------
# REACTANT INPUT PAGE
# ----------------------------------------

@route
def enter_reactants(state: State) -> Page:

    current_reactants = chemical_list(state, True)

    content = [
        Header("Enter Reactants"),

        "Enter the information for each reactant.",

        "You can repeat this process to add multiple reactants.",

        Header("Chemical Formula", 3),
        TextBox("chemical_name"),

        "Coefficient:",
        TextBox("coefficient", 1, "number"),

        "Molar Mass (grams per mole):",
        TextBox("molar_mass", 1.0, "number"),

        "Starting Mass (grams):",
        TextBox("mass", 1.0, "number"),

        Button("Save Reactant", "save_reactant"),

        Header("Current Reactants", 2)
    ]

    if len(current_reactants) > 0:
        content.append(BulletedList(current_reactants))
    else:
        content.append("No reactants entered yet.")
```

```
    return Page(state, content)


@route
def save_reactant(
        state: State,
        chemical_name: str,
        coefficient: int,
        molar_mass: float,
        mass: float) -> Page:

    if chemical_name.strip() == "":
        state.error_message = "Enter a chemical formula."
        return Page(state, [
            state.error_message,
            Link("Try again", "enter_reactants")
        ])

    if coefficient <= 0 or molar_mass <= 0 or mass < 0:
        state.error_message = "Enter valid chemical values."
        return Page(state, [
            state.error_message,
            Link("Try again", "enter_reactants")
        ])

    new_chemical = Chemical(
        chemical_name,
        coefficient,
        molar_mass,
        mass,
        True
    )

    state.chemicals.append(new_chemical)
    state.error_message = ""

    return Page(state, [
        Header("Reactant Saved!"),

        "You added " + chemical_name + " successfully.",

        Button("Add Another Reactant", "enter_reactants"),
```

```
        Button("Continue to Product", "enter_products"),

        Link("Home", "home")
    ])


# ----------------------------------------
# PRODUCT INPUT PAGE
# ----------------------------------------

@route
def enter_products(state: State) -> Page:
    return Page(state, [
        Header("Enter Product"),

        "Enter the chemical formula of your selected product.",

        "You can calculate one product at a time.",

        "Chemical Formula:",
        TextBox("chemical_name"),

        "Coefficient:",
        TextBox("coefficient", 1, "number"),

        "Molar Mass (grams per mole):",
        TextBox("molar_mass", 1.0, "number"),

        "Actual Product Mass Collected (grams):",
        TextBox("mass", 1.0, "number"),

        Button("Save Product", "save_product"),

        Link("Home", "home")
    ])


@route
def save_product(
        state: State,
        chemical_name: str,
        coefficient: int,
        molar_mass: float,
        mass: float) -> Page:
```

```
if chemical_name.strip() == "":
    state.error_message = "Enter a product formula."
    return Page(state, [
        state.error_message,
        Link("Try again", "enter_products")
    ])

if coefficient <= 0 or molar_mass <= 0 or mass < 0:
    state.error_message = "Enter valid product values."
    return Page(state, [
        state.error_message,
        Link("Try again", "enter_products")
    ])

product_found = False

# Update the existing product if one was entered.
for chemical in state.chemicals:
    if not chemical.is_reactant:
        chemical.name = chemical_name
        chemical.coefficient = coefficient
        chemical.molar_mass = molar_mass
        chemical.mass = mass
        product_found = True

# Otherwise, create a new product.
if not product_found:
    new_product = Chemical(
        chemical_name,
        coefficient,
        molar_mass,
        mass,
        False
    )

    state.chemicals.append(new_product)

state.actual_yield = mass
state.error_message = ""

return Page(state, [
    Header("Product Saved!"),
```

```
        "Your product information has been saved.",

        Button("Calculate Results", "results"),

        Button("Edit Product", "enter_products"),

        Link("Home", "home")
    ])


# ----------------------------------------
# RESULTS PAGE
# ----------------------------------------

@route
def results(state: State) -> Page:

    successful = calculate_values(state)

    if not successful:
        return Page(state, [
            Header("Calculation Error"),

            state.error_message,

            Link("Enter Reactants", "enter_reactants"),
            Link("Enter Product", "enter_products"),
            Link("Home", "home")
        ])

    return Page(state, [
        Header("Calculation Results"),

        Header("Chemical Equation", 2),
        state.equation,

        Header("Limiting Reactant", 2),
        state.limiting_reactant,

        Header("Theoretical Yield", 2),
        str(round(state.theoretical_yield, 4)) + " grams",

        Header("Actual Yield", 2),
        str(round(state.actual_yield, 4)) + " grams",
```

```
        Header("Percent Yield", 2),
        str(round(state.percent_yield, 2)) + "%",

        Button("How Were These Calculated?", "explanation"),

        Button("New Calculation", "clear_reaction"),

        Link("Home", "home")
    ])


# ----------------------------------------
# EXPLANATION PAGE
# ----------------------------------------

@route
def explanation(state: State) -> Page:

    successful = calculate_values(state)

    if not successful:
        return Page(state, [
            Header("Calculation Explanation"),

            "Enter your reaction information before "
            "viewing the calculation steps.",

            state.error_message,

            Link("Enter Equation", "enter_equation"),
            Link("Home", "home")
        ])

    return Page(state, [
        Header("How the Calculator Works"),

        Header("Step 1: Convert Grams to Moles", 2),

        "Divide each reactant's mass by its molar mass.",

        "Moles = Mass / Molar Mass",

        Header("Step 2: Find the Limiting Reactant", 2),
```

```
        "Divide each reactant's number of moles by "
        "its coefficient in the balanced equation.",

        "The reactant with the smallest ratio is the "
        "limiting reactant.",

        "Your limiting reactant is: " +
        state.limiting_reactant,

        Header("Step 3: Calculate Theoretical Yield", 2),

        "Multiply the smallest ratio by the product "
        "coefficient to find the theoretical moles "
        "of product.",

        "Then multiply product moles by the product "
        "molar mass to convert to grams.",

        "Theoretical Yield: " +
        str(round(state.theoretical_yield, 4)) +
        " grams",

        Header("Step 4: Calculate Percent Yield", 2),

        "Divide actual yield by theoretical yield "
        "and multiply by 100.",

        "Percent Yield: " +
        str(round(state.percent_yield, 2)) + "%",

        Button("View Results", "results"),

        Link("Home", "home")
    ])


# ----------------------------------------
# HELP PAGE
# ----------------------------------------

@route
def help(state: State) -> Page:
```

```
    return Page(state, [
        Header("Help and Instructions"),

        "Welcome to the Stoichiometry Calculator help page.",

        Header("How to Use the Website", 2),

        "1. Enter an already balanced chemical equation.",

        "2. Enter each reactant's formula, coefficient, "
        "molar mass, and starting mass.",

        "3. Add all reactants before continuing.",

        "4. Enter one product's formula, coefficient, "
        "molar mass, and actual mass collected.",

        "5. Select Calculate Results.",

        "6. Review the limiting reactant, theoretical "
        "yield, and percent yield.",

        Header("Important Information", 2),

        "Make sure your chemical equation is balanced.",

        "Enter coefficients from the balanced equation.",

        "Molar mass must be entered in grams per mole.",

        "All starting masses and product masses must "
        "be entered in grams.",

        "The chemical formulas and coefficients must "
        "match the balanced equation.",

        Button("Start a Calculation", "enter_equation"),

        Link("Home", "home")
    ])


# ----------------------------------------
# CLEAR / RESET ROUTE
```

```
# ----------------------------------------

@route
def clear_reaction(state: State) -> Page:

    state.equation = ""
    state.chemicals = []
    state.limiting_reactant = ""
    state.theoretical_yield = 0.0
    state.actual_yield = 0.0
    state.percent_yield = 0.0
    state.error_message = ""

    return index(state)


# ----------------------------------------
# UNIT TESTS
# ----------------------------------------

test_state = State(
    "2 H2 + O2 -> 2 H2O",
    [
        Chemical("H2", 2, 2.0, 2.0, True),
        Chemical("O2", 1, 32.0, 32.0, True),
        Chemical("H2O", 2, 18.0, 15.0, False)
    ],
    "",
    0.0,
    0.0,
    0.0,
    ""
)

assert calculate_values(test_state)
assert test_state.limiting_reactant == "H2"
assert round(test_state.theoretical_yield, 2) == 18.0
assert round(test_state.actual_yield, 2) == 15.0
assert round(test_state.percent_yield, 2) == 83.33


# ----------------------------------------
# START WEBSITE
# ----------------------------------------
```

```
start_server(State(
    "",
    [],
    "",
    0.0,
    0.0,
    0.0,
    ""
))
```